COMPUTER NETWORKS

TEAM NETWORKS

Department of Computer Science and Engineering

COMPUTER NETWORKS

Link Layer and LAN

TEAM NETWORKS

Department of Computer Science and Engineering

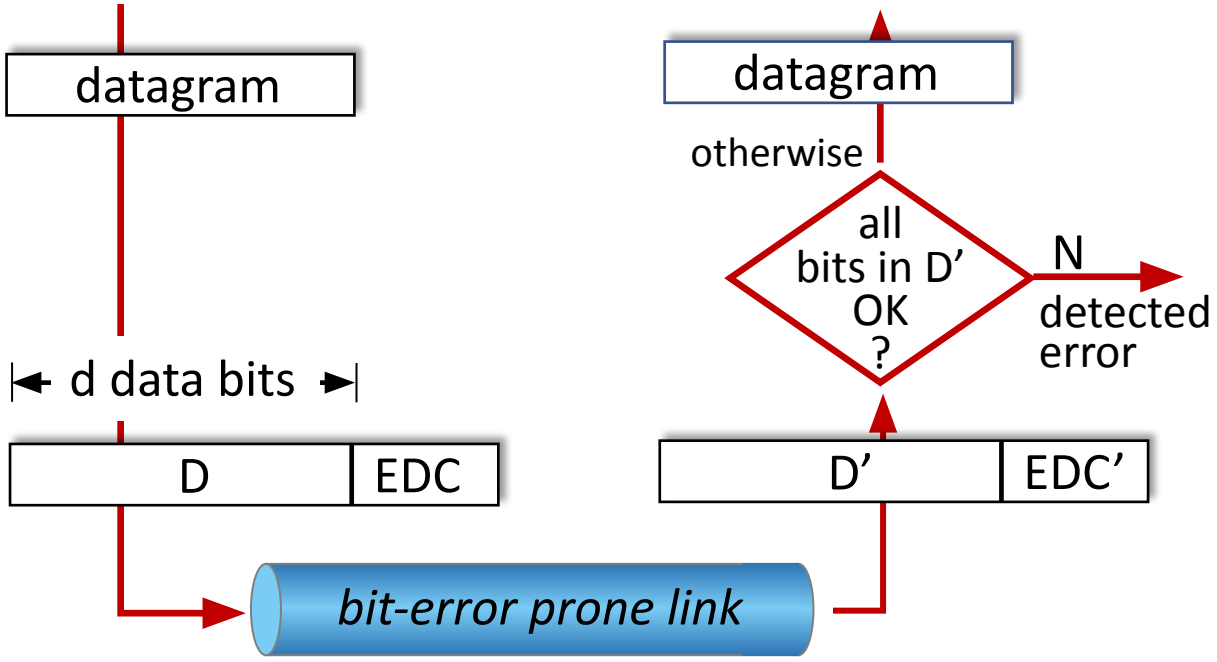
COMPUTER NETWORKS

Unit – 4 Link Layer and LAN Roadmap

- Introduction
- Error detection, correction
- Multiple access protocols
- LANs
  - addressing, ARP
  - Ethernet
  - switches
- A day in the life of a web request
- Physical layer
- Wireless LANs: IEEE 802.11

Error detection

EDC: error detection and correction bits (e.g., redundancy)
D: data protected by error checking, may include header fields



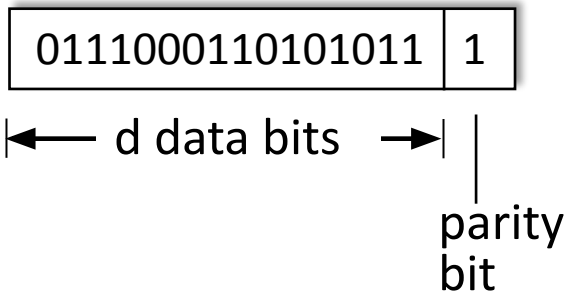
Error detection not 100% reliable!

- protocol may miss some errors, but rarely
- larger EDC field yields better detection and correction

Parity checking

single bit parity:

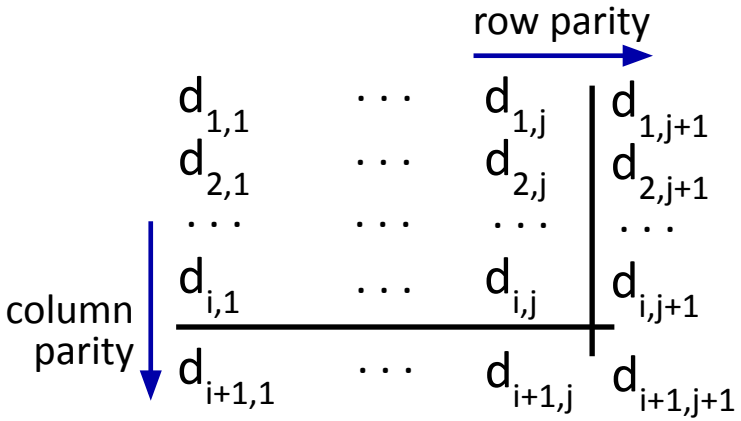
- detect single bit errors



Even parity: set parity bit so there is an even number of 1's

two-dimensional bit parity:

- detect *and correct* single bit errors



no errors:

1	0	1	0	1	1
1	1	1	1	0	0
0	1	1	1	0	1
0	0	1	0	1	0

detected and correctable single-bit error:

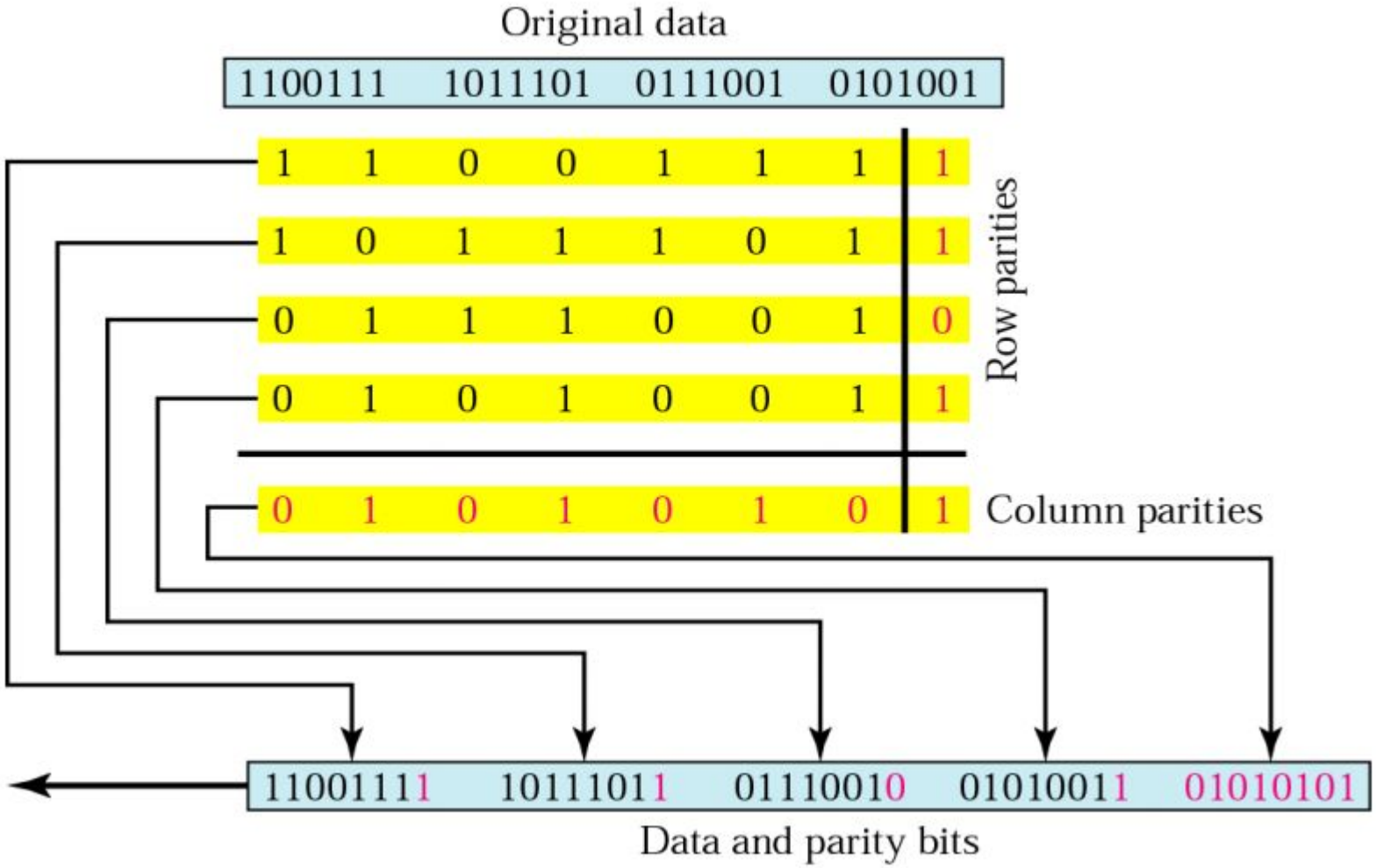
1	0	1	0	1	1
1	0	1	1	0	0
0	1	1	1	0	1
1	0	1	0	1	0

parity error

parity error

COMPUTER NETWORKS

Two dimensional bit parity check - Example



Internet checksum (review)

Goal: detect errors (*i.e.,* flipped bits) in transmitted segment

Sender:

- treat contents of UDP segment (including UDP header fields and IP addresses) as sequence of 16-bit integers
- checksum: addition (one's complement sum) of segment content
- checksum value put into UDP checksum field

Receiver:

- compute checksum of received segment
- check if computed checksum equals checksum field value:
  - not equal - error detected
  - equal - no error detected. *But maybe errors nonetheless?* More later ….

COMPUTER NETWORKS

Two dimensional bit parity check - Example

0110011001100110
0101010101010101
0000111100001111 } Three 16 bits words

The sum of first of two 16-bit words is:

0110011001100110
0101010101010101

1011101110111011 → Sum of 1st two 16 bit words.

Adding the third word to the above sum gives:

1011101110111011 → Sum of 1st two 16 bit words.
0000111100001111 → Third 16 bits word

1100101011001010 → Sum of all three 16 bit words.

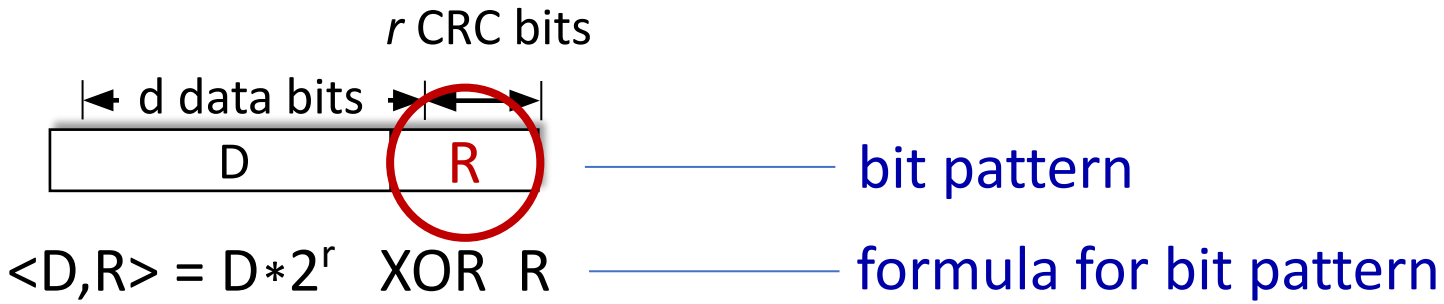
Taking 1's complement for the final sum:

1100101011001010 → Sum of all three 16 bit words.
0011010100110101 → 1's complement for the final sum.

The 1's complement value is called as **Checksum**

Cyclic Redundancy Check (CRC Codes)

- more powerful error-detection coding
- D: data bits (given, think of these as a binary number)
- G: bit pattern (generator), of *r+1* bits (given)



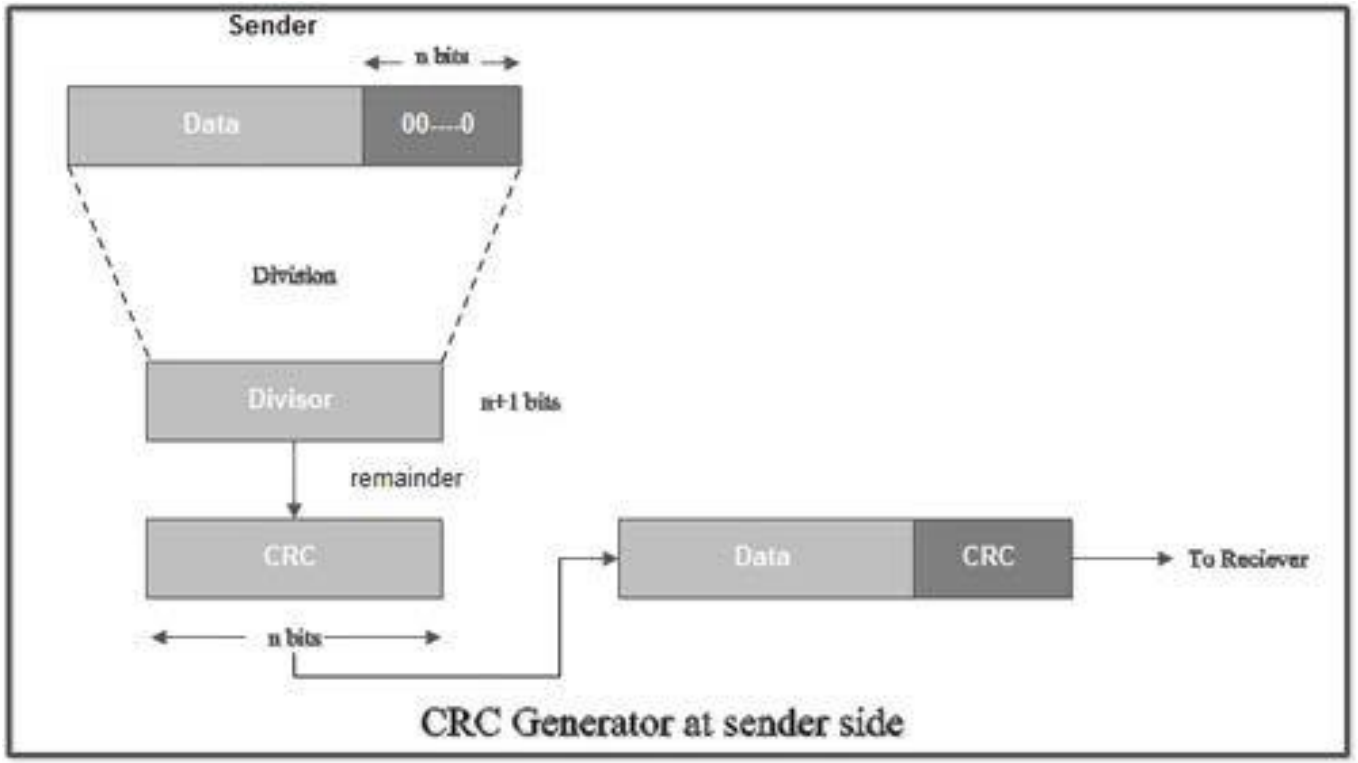
goal: choose *r* CRC bits, R, such that <D,R> exactly divisible by G (mod 2)

- receiver knows G, divides <D,R> by G. If non-zero remainder: error detected!
- can detect all burst errors less than r+1 bits
- widely used in practice (Ethernet, 802.11 WiFi)

COMPUTER NETWORKS

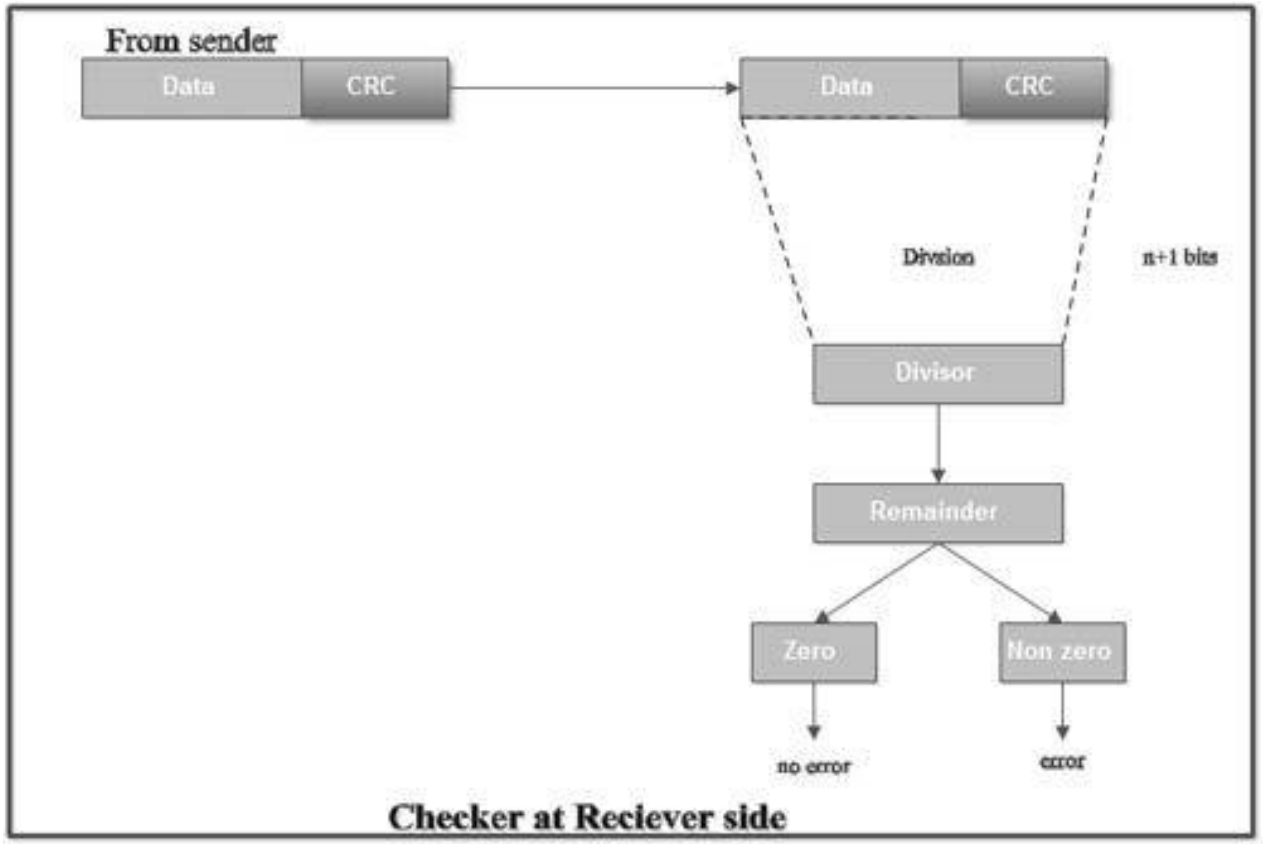
Sender Side – Calculation of CRC

- A string of n 0's is appended to the data unit to be transmitted.
- Here, n is one less than the number of bits in CRC generator.
- Binary division is performed of the resultant string with the CRC generator.
- After division, the remainder so obtained is called as CRC.
- It may be noted that CRC also consists of n bits.



Receiver Side – Check

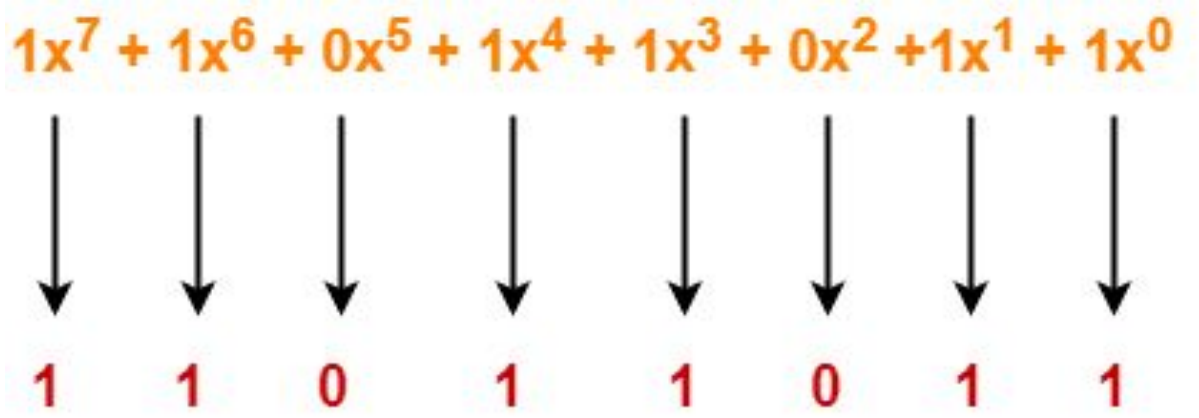
- The received code word is divided with the same CRC generator.
- On division, the remainder so obtained is checked.
- **Case-1: Remainder = 0,**
  - No error
  - Receiver accepts the data
- **Case-2: Remainder ≠ 0**
  - Receiver assumes that some error occurred
  - Receiver rejects the data and asks the sender for retransmission.



COMPUTER NETWORKS

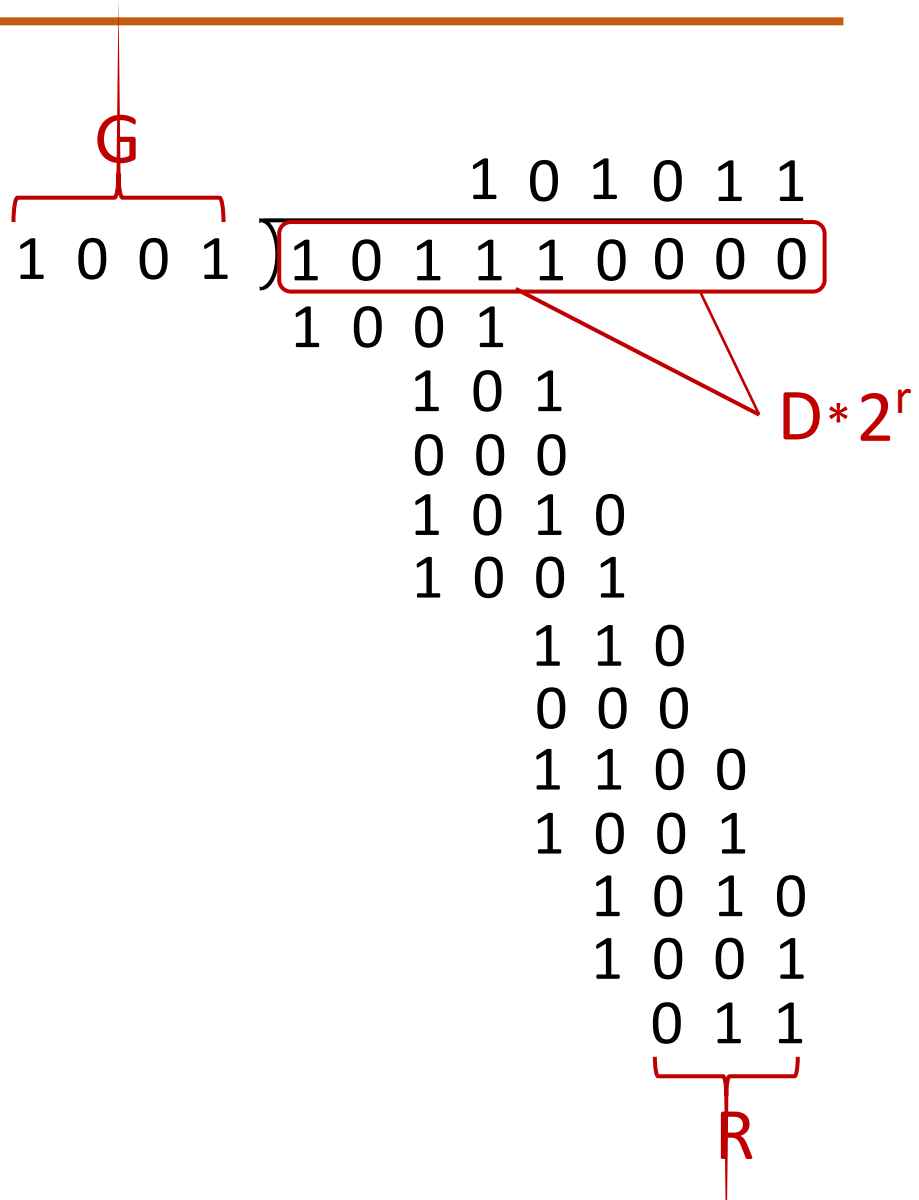
CRC Generator

- an algebraic polynomial represented as a bit pattern
- Rule - The power of each term gives the position of the bit and the coefficient gives the value of the bit.
- Consider the CRC generator is $x^7 + x^6 + x^4 + x^3 + x + 1$.



Cyclic Redundancy Check (CRC): Example

Calculation for the case of:
D = 101110, d = 6,
G = 1001, and r = 3.

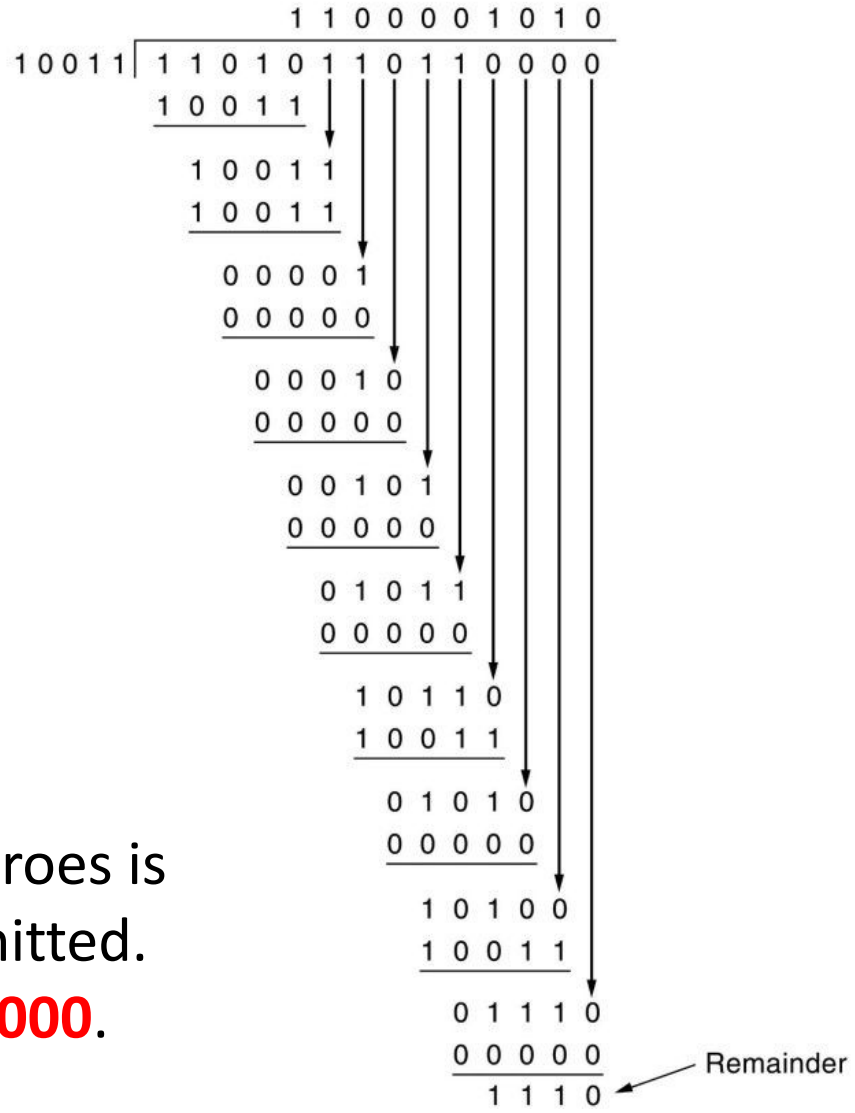


COMPUTER NETWORKS

CRC – Numerical Example

A bit stream 1101011011 is transmitted using the standard CRC method. The generator polynomial is x^4+x+1.

What is the actual bit string transmitted?



Solution

- ✔ Generator polynomial $G(x) = x^4 + x + 1$ is encoded as **10011**.
- ✔ G(x) consists of 5 bits. So, a string of 4 zeroes is appended to the bit stream to be transmitted.
- ✔ The resulting bit stream is 1101011011**0000**.

CRC – Numerical Example

- A bit stream 10011101 is transmitted using the standard CRC method. The generator polynomial is x^3+1.
  - What is the actual bit string transmitted?
  - Suppose the third bit from the left is inverted during transmission. How will receiver detect this error?

Solution

✔ Generator polynomial G(x) = x^3 + 1 is encoded as **1001**.

✔ G(x) consists of 4 bits. 4 zeroes is appended now.

✔ The resulting bit stream is 10011101**000**.

Solution (Part-1)

✔ Code word to be transmitted is obtained by replacing the last 3 zeroes of 10011101**000** with the CRC.

✔ Code word transmitted to the receiver = 10011101**100**.

```
          10001100
      ┌──────────
1001  │ 10011101000
        1001
        -----
        00001
        0000
        ----
        00011
        0000
        ----
        00110
        0000
        ----
        01101
        1001
        ----
        01000
        1001
        ----
        00010
        0000
        ----
        00100
        0000
        ----
        0100   ← CRC
```

CRC – Numerical Example

Solution (Part-2)

According to the question,

- ✔ The bit stream received by the receiver = 10111101100.
- ✔ Receiver performs the binary division with the same G(x).
- ✔ Remainder obtained on division is a non-zero value.
- ✔ Error occurred

```
          10101000
      ┌──────────
1001  │ 10111101100
        1001
        -----
        00101
        0000
        -----
        01011
        1001
        -----
        00100
        0000
        -----
        01001
        1001
        -----
        00001
        0000
        -----
        00010
        0000
        -----
        00100
        0000
        -----
        0100   ← Remainder
```

THANK YOU

TEAM NETWORKS

Department of Computer Science and Engineering